ManyTutorial 10
Mongo DB
(The Little MongoDB Book by Karl Seguin)

Task 1: Installing MongoDB and making a connection (On Your machines)

Refer to the lecture 10 slides (31-35).

Task 2: We will create a collection or table (Chapter 1)

1. Create a collection (or table) called unicorns

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
db.unicorns.find()
```

```
test> db.version()
6.0.0
test> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
{
  acknowledged: true,
  insertedId: ObjectId("63364d9329c2226d243e8336")
}
test> db.unicorns.find()
[
  {
    _id: ObjectId("63364d9329c2226d243e8336"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
test>
```

2. Add another document (or row) as

```
db.unicorns.insertOne({name: "Leto", gender: "m", home: "Arrakeen", worm: false})
```

3. Use find() method (or select command)

```
db.unicorns.find()
```

You should see two documents (rows)

```
test> db.unicorns.find()
[
  {
    _id: ObjectId("63364d9329c2226d243e8336"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("63364e3329c2226d243e8337"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false
  }
]
```

4. Now add the following more documents

```
db.unicorns.insertOne({name: "Horny", dob: new Date(1992,2,13,7,47),
loves:["carrot","papaya"],weight: 600, gender: "m", vampires: 63});

db.unicorns.insertOne({name: "Aurora", dob: new Date(1991, 0, 24, 13, 0), loves:
["carrot", "grape"], weight: 450, gender: "f", vampires: 43});

db.unicorns.insertOne({name: "Unicrom", dob: new Date(1973, 1, 9, 22, 10), loves:
["energon", "redbull"], weight: 984, gender: "m", vampires: 182});

db.unicorns.insertOne({name: "Roooooodles", dob: new Date(1979, 7, 18, 18, 44),
loves: ["apple"],weight: 575, gender: "m", vampires: 99});

db.unicorns.insertOne({name: "Solnara", dob: new Date(1985, 6, 4, 2, 1),
loves:["apple", "carrot", "chocolate"], weight:550, gender: "f", vampires:80});

db.unicorns.insertOne({name:"Ayna", dob: new Date(1998, 2, 7, 8, 30), loves:
["strawberry", "lemon"], weight: 733, gender: "f", vampires: 40});

db.unicorns.insertOne({name:"Kenny", dob: new Date(1997, 6, 1, 10, 42), loves:
["grape", "lemon"],weight: 690, gender: "m", vampires: 39});

db.unicorns.insertOne({name: "Raleigh", dob: new Date(2005, 4, 3, 0, 57), loves:
["apple", "sugar"],weight: 421, gender: "m", vampires: 2});

db.unicorns.insertOne({name: "Leia", dob: new Date(2001, 9, 8, 14, 53), loves:
["apple", "watermelon"], weight: 601, gender: "f", vampires: 33});

db.unicorns.insertOne({name: "Pilot", dob: new Date(1997, 2, 1, 5, 3), loves:
["apple", "watermelon"], weight: 650, gender: "m", vampires: 54});

db.unicorns.insertOne({name: "Nimue", dob: new Date(1999, 11, 20, 16, 15), loves:
["grape", "carrot"], weight: 540, gender: "f"});

db.unicorns.insertOne({name: "Dunx", dob: new Date(1976, 6, 18, 18, 18), loves:
["grape", "watermelon"], weight: 704, gender: "m", vampires: 165});
```

5. Use find() method (or select command) again

```
db.unicorns.find()
```

Task 3: Using Selectors (Chapter 1)

1. Using the find method

```
db.unicorns.find({gender: "m", weight: {$gt: 700}})
```

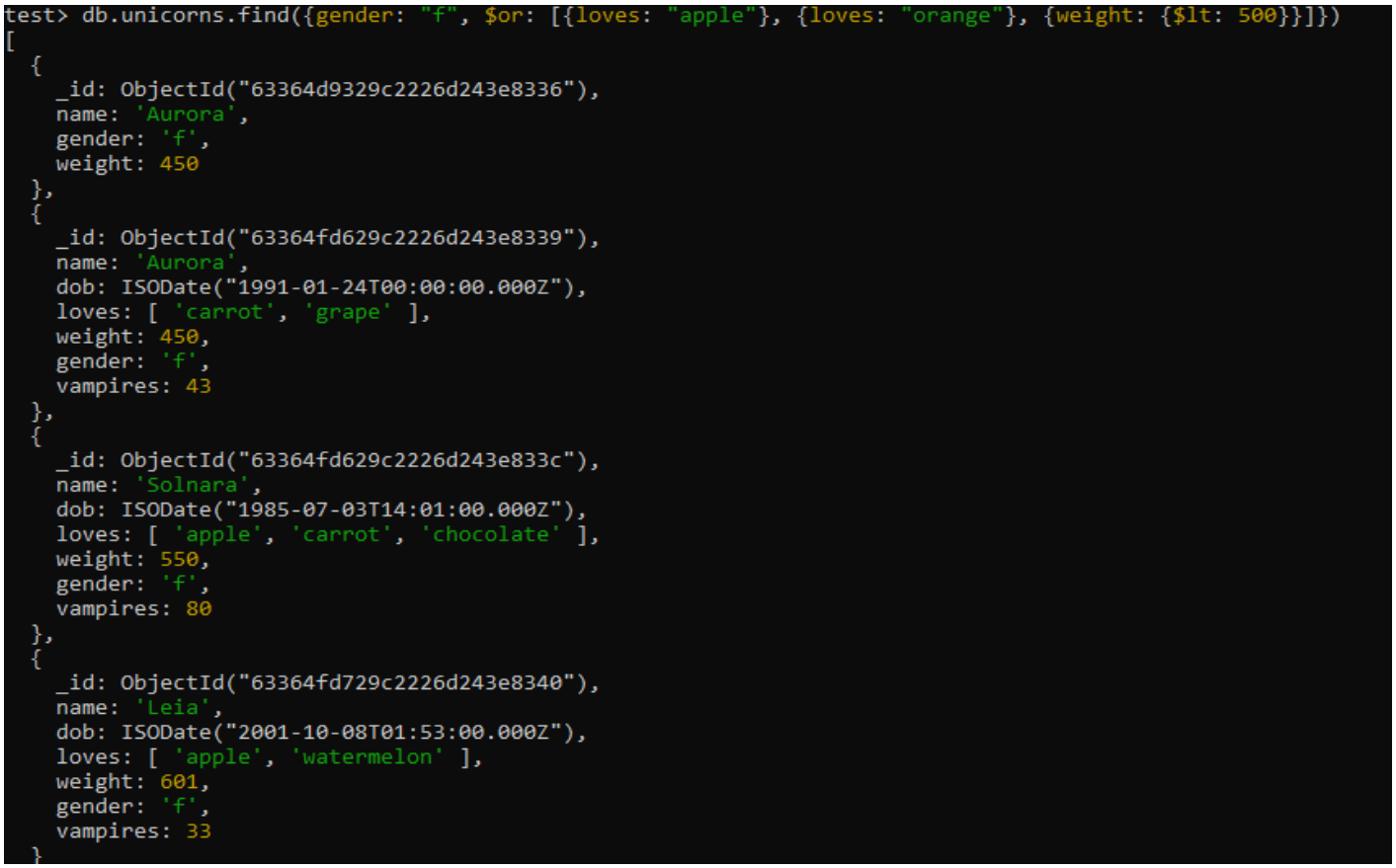
```
db.unicorns.find({gender: {$ne: "f"}, weight: {$gte: 704}})
```

```
test> db.unicorns.find({gender: "m", weight: {$gt: 700}})
[
  {
    _id: ObjectId("63364fd629c2226d243e833a"),
    name: 'Unicrom',
    dob: ISODate("1973-02-09T10:10:00.000Z"),
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId("6336500029c2226d243e8343"),
    name: 'Dunx',
    dob: ISODate("1976-07-18T06:18:00.000Z"),
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
test> db.unicorns.find({gender: {$ne: "f"}, weight: {$gte: 704}})
[
  {
    _id: ObjectId("63364fd629c2226d243e833a"),
    name: 'Unicrom',
    dob: ISODate("1973-02-09T10:10:00.000Z"),
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId("6336500029c2226d243e8343"),
    name: 'Dunx',
    dob: ISODate("1976-07-18T06:18:00.000Z"),
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

```
db.unicorns.find({vampires: {$exists: false}})
```

```
test> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId("63364d9329c2226d243e8336"),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  },
  {
    _id: ObjectId("63364e3329c2226d243e8337"),
    name: 'Leto',
    gender: 'm',
    home: 'Arrakeen',
    worm: false
  },
  {
    _id: ObjectId("6336500029c2226d243e8342"),
    name: 'Nimue',
    dob: ISODate("1999-12-20T03:15:00.000Z"),
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

db.unicorns.find({gender: "f", $or: [{loves: "apple"}, {loves: "orange"}, {weight: {$lt: 500}}]})



Task 4: Updating (Chapter 2) and Selecting (Chapter 3)

1. To find the object id of “Roooooodles”
   db.unicorns.find({name:"Roooooodles"})

```
test> db.unicorns.find({name:"Roooooodles"})
[
  {
    _id: ObjectId("63364fd629c2226d243e833b"),
    name: 'Roooooodles',
    dob: ISODate("1979-08-18T06:44:00.000Z"),
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  }
]
```

2. To update Roooooodles with correct weight
   db.unicorns.updateOne({name: "Roooooodles"}, {$set: {weight: 590}})

3. Other update modifiers are $inc, $push
   db.unicorns.updateOne({name: "Pilot"}, {$inc: {vampires: -2}})
   db.unicorns.updateOne({name: "Aurora"}, {$push: {loves: "sugar"}})

```
test> db.unicorns.updateOne({name: "Pilot"}, {$inc: {vampires: -2}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>

test> db.unicorns.updateOne({name: "Aurora"}, {$push: {loves: "sugar"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

4. Make multiple updates to insert another attribute
   db.unicorns.updateMany({}, {$set: {vaccinated: true }}, false, true);
   db.unicorns.find({vaccinated: true});

5. To find a list of names
   db.unicorns.find(null, {name: 1});

```
test> db.unicorns.find(null, {name: 1});
[
  { _id: ObjectId("63364d9329c2226d243e8336"), name: 'Aurora' },
  { _id: ObjectId("63364e3329c2226d243e8337"), name: 'Leto' },
  { _id: ObjectId("63364fd629c2226d243e8338"), name: 'Horny' },
  { _id: ObjectId("63364fd629c2226d243e8339"), name: 'Aurora' },
  { _id: ObjectId("63364fd629c2226d243e833a"), name: 'Unicrom' },
  { _id: ObjectId("63364fd629c2226d243e833b"), name: 'Roooooodles' },
  { _id: ObjectId("63364fd629c2226d243e833c"), name: 'Solnara' },
  { _id: ObjectId("63364fd629c2226d243e833d"), name: 'Ayna' },
  { _id: ObjectId("63364fd729c2226d243e833e"), name: 'Kenny' },
  { _id: ObjectId("63364fd729c2226d243e833f"), name: 'Raleigh' },
  { _id: ObjectId("63364fd729c2226d243e8340"), name: 'Leia' },
  { _id: ObjectId("63364fd729c2226d243e8341"), name: 'Pilot' },
  { _id: ObjectId("6336500029c2226d243e8342"), name: 'Nimue' },
  { _id: ObjectId("6336500029c2226d243e8343"), name: 'Dunx' }
]
```

6. To find a list of names without the _id
   db.unicorns.find(null, {name: 1, _id:0});

7. Order by weight descending
   db.unicorns.find().sort({weight: -1})

8. Order by weight descending and display just name and weight
   db.unicorns.find(null, {name: 1, weight: 1, _id:0}).sort({weight: -1})

9. Order by unicorn name, then vampire kills
   db.unicorns.find().sort({name: 1, vampires: -1})

10. Count all vampires > 50
    db.unicorns.countDocuments({vampires: {$gt: 50}})
    OR
    db.unicorns.find({vampires: {$gt: 50}}).count()

Some tricks to work on the Console Prompt

1. To Paste something
   Press Alt and Spacebar together. Then use shortcut menus E (edit) and P (paste)

2. To clear the screen
   cls

3. To come back to the cursor and break the current process
   Press Ctrl and C together

4. To exit from the Mongo shell
   Exit

   Close the server command prompt screen to close the server after exiting Mongo shell.

Tutorial Exercise

Run the following SQL statements in SQL Developer:

```
CREATE TABLE PATIENTMEDICALRECORD (
TEST_ID INTEGER PRIMARY KEY,
TEST_DATE        DATE,
PAT_LNAME        VARCHAR (15) NOT NULL,
PAT_FNAME        VARCHAR (15),
HDL_CHOL         NUMBER,
LDL_CHOL         NUMBER,
TRIGLYCER        NUMBER,
GLUCOSE          NUMBER
);

ALTER SESSION SET NLS_DATE_FORMAT = 'DD/MM/YYYY';


/* PATIENT MEDICAL TEST RECORD rows                                    */

INSERT INTO PATIENTMEDICALRECORD VALUES (1, '23/5/2017','Kolmycz','George', 45, 120, 154, NULL);
INSERT INTO PATIENTMEDICALRECORD VALUES (2, '14/6/2017','Lewis','Rhonda', NULL, 129, NULL, 108);
INSERT INTO PATIENTMEDICALRECORD VALUES (5, '24/9/2016','Vandam', NULL, NULL, 123, NULL, 112);
INSERT INTO PATIENTMEDICALRECORD VALUES (6, NULL, 'Jones', 'Anne', NULL, NULL, NULL, NULL);

COMMIT;
```

1. Convert the above to a collection PATIENTDATA in MongoDB.
2. Write SQL statements (for PATIENTMEDICALRECORD) and NoSQL statements (for PATIENTDATA) for the following:
   i. Select all the patients' medical records from PATIENTMEDICALRECORD and PATIENTDATA.
   ii. Update Anne Jones medical records (Test Date = 1st Oct 2022 and Glucose level = 99).
   iii. Select all patients having glucose level less than 100.
   iv. Increment Vandam's glucose level by 5 (i.e., = 117). You should increment in both SQL and NoSQL (not add a new value).
   v. Add an attribute NZResident = true to all the rows in the table/collection.
   vi. Change Anne Jones NZResident status to false.
   vii. Sort the patient records by their last names in descending order.
   viii. Count all patients having glucose level > 100 and ldl_chol >= 123.
   ix. Remove Anne Jones from the list.
   x. List all patient names with their glucose levels (with glucose levels in descending order). Hide the ObjectId for NoSQL collection.